OS-Integrated Buffer Management for High-Throughput Analytical Workloads

Maurice Scholtes
maurice.scholtes@tum.de
Technical University of Munich

Abstract

Cloud data systems are becoming increasingly resource- and system-disaggregated. An ecosystem of heterogeneous applications communicating via open table formats (OTFs) and object storage has placed renewed importance on latency-hiding caching interfaces for analytics systems.

We propose a new type of buffer manager which exploits the processor's virtual memory subsystem to enable fragmentation-free caching of dynamically-sized logical file regions. We develop novel TLB management and latency-masking techniques which allow our buffer manager to scale to many-core systems, with the aim of integrating our buffer manager into a broader project to memory-map files from object storage over the network.

Our evaluation shows that file format-aware buffer management and eviction policies achieve a speedup of up to 2.90× when compared to existing solutions across all 22 TPC-H queries at the same cache size. Our IPI-free TLB invalidation technique demonstrates increasing speedups with core counts, achieving 6.7% lower query latency at 64 cores when compared to our baseline.

1 Introduction

Variably sized data is everywhere [19]. In strings, file systems, media files, object stores, as a result of compression, and in web caches. While an extensive body of research on caching policies and buffer management exists, the focus remains mostly on the caching of fixed-sized elements. Comparatively, limited research exists on cache management for variably sized data.

Cloud data analytics systems employ caches to hide the latency of accessing data files over the network. An increasingly disaggregated cloud landscape has resulted in increased access latencies, placing a greater emphasis on workload-aware interfaces to avoid inefficiencies.

As part of a broader project to enable memory-mapped access to network-remote object stores, we propose a buffer management solution for variably sized chunks within files. Specifically, we design and evaluate a file-format aware, virtual-memory based buffer manager which manages the caching of Parquet file regions on a logical level.

In doing so, we demonstrate techniques which enable our buffer manager to solve the thread-scaling and fragmentation challenges which are persistent problems in existing research.

Our design builds on top of uCache [13] and the OSv unikernel [9], and is evaluated against existing fixed-size file caching solutions.

We contribute the following:

1. A buffer manager solution that allows readers to access partially loaded buffers.
2. An approach to reconcile logical region-based eviction policies with fixed-sized memory frames.
3. A novel approach to TLB entry invalidation without TLB shootdowns.

2 Background

In this section, we introduce some of the operating system and database system concepts which inform our design.

2.1 Database Buffer Management

Modern database system typically do not use a buffer manager for ensuring data durability. Instead, the role of the buffer manager is to provide an interface for the query engine to access data stored out-of-memory. Traditionally, this has meant locally attached disk or SSD storage, but increasingly database systems are tasked with querying data over the network [4].

In such a scenario, the role of a buffer manager becomes increasingly relevant. I/O operations are cached so that repeated accesses to the same buffer can avoid latency. To achieve this, buffer managers must allocate, search for, concurrently protect, and evict/replace cached buffers from memory [7]. This role typically involves the translation of logical region

or buffer identifiers to virtual, and then physical memory addresses.

2.2 Parquet

Apache Parquet[1] is an open source, columnar file format commonly used for data storage and exchange.

The Parquet format includes several features designed to improve analytical performance. We highlight the following relevant properties.

Row group and column chunk partitioning. Parquet partitions the stored data in row groups, which then contain column chunks of contiguous data stored on logical pages. This enables systems to skip scanning entire row groups or unused columns.

Compression and Encoding. The columnar data stored on Parquet pages can be encoded and additionally compressed with a generic compression scheme. Query engines can implement a technique known as predicate push-down to evaluate predicates without decoding the data [12]. The use of compression and encoding schemes also means that the sizes of each logical region within a Parquet file can vary.

Bloom filters and page statistics. Parquet pages include optional bloom filters and statistics on several granularities which allow query engines to skip over data ranges.

Footer. The Parquet format includes metadata about a file's layout in the footer. This data can be cached by data systems, allowing them to access only the relevant subsections of a Parquet file.

2.3 Memory-Mapped I/O

The memory-mapped I/O interface presents a mapped file resource as a region of continuous memory to the program, as-if the file resides in-memory. This is enabled through the use of virtual memory.

When the mapping is established, the operating system populates a virtual memory area (VMA) with page table entries (PTEs). These pages may be guarded by certain flags (e.g., present bit) in their corresponding PTEs. When the application attempts to access a guarded page, hardware processor mechanisms trigger a page fault, which switches the execution context to a page fault handler. This fault handler is responsible for satisfying the corresponding I/O request before execution is returned to the application code.

Because not all hardware memory frames need to be allocated as backing memory for a VMA, memory-mapped I/O interfaces often act as a cache for masking the latency of the underlying I/O resource. The operating system makes decisions about when reads or writes to a memory-mapped file are evicted or flushed.

[1] https://parquet.apache.org/

Memory-mapped I/O interfaces offered by OSs often suffer from poor performance [5]. The available general purpose eviction and prefetching policies are not attunable to workload-specific characteristics that applications are privy to. Additionally, the need to present a coherent virtual memory interface to clients results in TLB shootdowns, which do not scale to many-core machines (section 2.4).

2.4 Translation lookaside buffers (TLB)

Both ARM and `x86_64` architecture implementations include hardware TLBs, which are used to cache the translation from a virtual address to a physical one on a per-core basis.

TLB Invalidation. When page tables are updated (particularly when a page frame is unmapped), operating systems need to invalidate the TLB entries of all relevant cores. This is known as a TLB shootdown, and is typically achieved using an inter-processor interrupt (IPI). The evicting core sends a hardware interrupt to all other cores, which must: (1) switch execution context, (2) invalidate local TLB entries, and then (3) notify the evicting core of completion, before continuing with their execution.

Systems (e.g., Linux, vmcache) typically amortize this overhead by batching eviction and TLB invalidations.

3 Design

Our design leverages application-specific file format awareness to inform OS-level memory management decisions. We extend the memory-mapped file interface by allowing the application to inform physical page frame allocation and eviction. This allows our application to implement buffer manager decisions on the dynamically-sized logical regions of file being mapped, instead of relying on the 4 KiB page frame granularity offered by the hardware and OS.

Our design leverages the memory-mapping interface for logical region *translation*, i.e., the conversion from a logical id to a physical address. Despite similarities to memory-mapped files, memory access to the virtual memory area (VMA) without participating in our design's synchronization scheme is not supported.

This approach allows our design to avoid the primary problem associated with variable size buffer management: memory fragmentation. We exploit the the processor's virtual memory subsystem to allocate and free the underlying physical frames from any virtual memory location without relocation. This results in the maximum fragmentation overhead being bounded by our frame granularity of 4 KiB on both sides of each buffer region. This approach of using the memory-mapped interface also simplifies allocation, as every logical region inherently has a fixed range in the VMA, and any frame can be used to back any logical page.

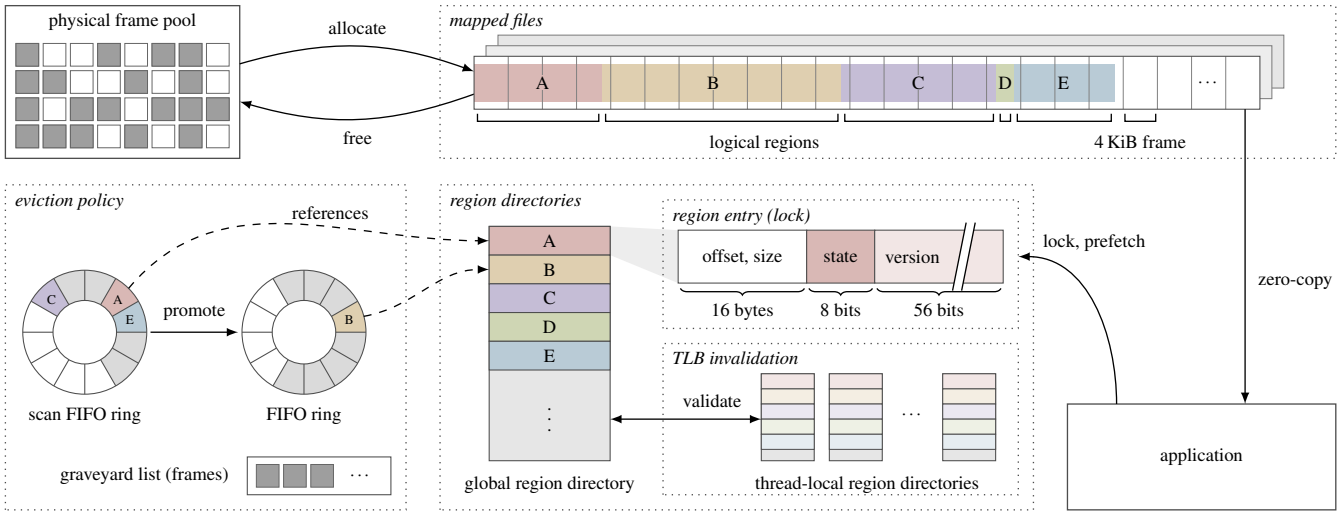


Figure 1: Design overview.

3.1 File Region Directory

Applications which intend to perform buffer managed access to a file need to first identify the relevant discrete logical file regions within the file. This can be achieved by reading a file format's header/footer, which typically contains the offset and length of each region. These regions within the file do not need to be aligned, of a minimum size, or adjacent, allowing our design to support file formats which were not designed for caching or direct I/O access.

Similarly to vmcache [11], our design then builds an array of `RegionEntry` locks in a global region directory, which allows the application to perform the translation from a logical region identifier into a virtual memory offset into the mapped file's VMA. We improve the processor cache locality of our design by bundling the lock with the region's file offset and size.

These locks are stored non-intrusively in a directory outside of the file's VMA, avoiding the need for mapped file formats to support a gap in their layout for an intrusive buffer header.

This global lock array is used for locking, prefetching, eviction, and our TLB shootdown-free memory unmapping scheme.

3.2 Locking Scheme

In order to prevent concurrent eviction while a file region is being read, our design introduces a shared/exclusive locking scheme inspired by the hybrid-lock introduced by Böttcher [3] and used by vmcache [11].

While our lock does contain a version counter, our design does not support optimistic `seqlock`-like optimistic reads into the buffer regions [10]. Instead, we use the version counter in our lock to signal TLB entry invalidation (Section 3.8).

Lock Layout. Our design uses a 64-bit atomic variable, which is read and updated using atomic load and compare-and-swap operations. Eight of the bits are used to represent the lock's state, and the remaining 56 bits are used for the TLB invalidation version counter.

We have a total of 5 states, which we show in figure 2. In addition to the 5 states, we also reserve one bit to signal if a region has been re-accessed (R). The remaining representable values in the in the state are used to count the number of shared lock holders (C).

The decision to use the remaining 56 bits for the version is arbitrary; the counter only needs to be large enough to avoid the "ABA problem", whereby the counter value wraps around back to the same value between reads, leading to a missed TLB invalidation [14]. We elaborate further on this mechanism in section 3.8.

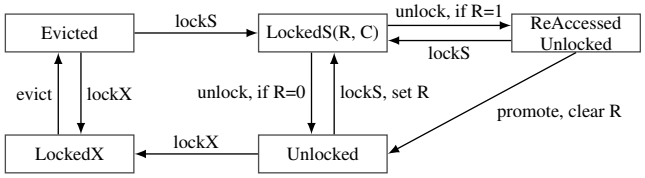


Figure 2: Logical region lock states and transitions.

Lock States. We initialize the lock array with all locks in the evicted state. Readers which wish to read from a region must pin it by acquiring a shared lock by incrementing the shared counter for the region.

When regions are unlocked, the the thread which sets the shared count (C) back to 0 transitions the lock state into either the unlocked or re-accessed unlocked state, depending on

whether or not the re-accessed bit (R) is set. The re-accessed bit is used to inform the eviction policy of regions which have been accessed multiple times (section 3.3), and is set by readers which acquire a shared lock from the unlocked state. The thread which releases the lock into the re-accessed unlocked state may differ from the one which set the re-accessed bit.

When evicting a region, we ensure that there are no concurrent readers accessing the region by taking an exclusive lock. This lock is released after eviction.

Our design additionally supports directly exclusively locking regions from either unlocked state, but this is not needed for read-only workloads.

3.3 Eviction Policy

Our design allows us to define eviction policies based on logical file regions, instead of needing to implement policy based on frames. This allows our design to avoid pathological eviction behavior (thrashing), whereby pages from multiple logical regions are partially evicted. Additionally, applications can specialize the eviction policy based on workload-specific semantics to a much greater degree, for example by weighing different logical regions differently. How the logical eviction policy is reconciled with the physical frame mapping is described in section 3.4.

Eviction is triggered when our backing physical frame allocator does not have enough available frames to satisfy the pinning of a new region. Because we evict logical regions instead of frames, our eviction policy greedily selects regions for eviction until the desired evicted frame count is available. During eviction, we acquire exclusive locks on the evicted regions to ensure that no readers or other threads may attempt to concurrently access/evict the region.

A resident set of in-memory logical file regions is used to ensure that the eviction policy remains efficient, even when operating on large files with many regions. This resident set contains entries of $\langle region_id, file_id \rangle$, which also solves the challenges associated with victim selection across multiple files.

Our prototype design specializes the eviction policy for an analytical Parquet file scan workload. This workload is characterized by a large number of regions which are accessed just once, and a small number of regions which are accessed repeatedly. Section 4 elaborates on the workload-specific elements of the implementation.

Our design uses a simplified, 2Q-like [8], S3-FIFO-inspired [20], resident set for eviction. Two FIFO ring buffer queues are used for the resident set.

Regions which are added to the resident set are first added to the tail of the scan queue. Eviction victims are selected from the scan queue's head. During lock acquisition, if the re-accessed bit (R) of the victim region's lock is set, we promote the victim to the main queue instead of evicting it (figure 2). When the scan queue is empty, we instead select a victim from the main queue.

Algorithm 1 Eviction procedure

```
1:  function EVICT_REGIONS(R) ▷ R: list of locked regions
2:      EVICT_GRAVEYARD_FRAMES
3:      sort(R)                  ▷ groups regions in the same file
4:      while R is not empty do
5:          r ← next region from R
6:          if r is adjacent to current run then
7:              extend run with r
8:          else
9:              EVICT_CONTIGUOUS_REGIONS(run)
10:             start a new run at r
11:         end if
12:     end while
13:     EVICT_CURRENT_RUN(run)
14:     release region locks
15: end function
```

Resident set sizing. In contrast to most cache and resident set designs, our set entries do not have a fixed physical frame size. This complicates typical cache sizing threshold and eviction high/low watermark selection. Our design does not enforce a static partitioning of the number of elements in each resident set queue. Instead, we allow the number of resident set entries (regions) in each queue to vary dynamically, but introduce accounting to translate resident regions back to a frame count.

3.4 Eviction Procedure

While our eviction policy selects logical *regions* for eviction, our VMA's virtual memory is backed by, and thus must reclaim, physical page *frames*. Our design uses fixed 4 KiB frames, but must support (possibly adjacent) logical regions of arbitrary size and file offset. In general, this means that the first and last backing frames of a given region may overlap with one (or multiple) other regions.

The regions selected by the eviction policy (section 3.3) are locked and passed to the eviction procedure. We describe the procedure to evict the underlying frames from the given set of regions in (simplified) algorithms 1 and 2.

The regions for eviction are sorted and then converted into contiguous runs of regions (in virtual memory) for eviction (algorithm 1). Algorithm 2 describes the procedure for evicting a run of regions/frames.

Region-Frame Reconciliation. Frames which span multiple regions (at the tail or head of the currently evicted region) must ensure that there are no readers or conflicting evicting threads accessing the overlapping frame. Since our design supports regions smaller than a frame (region D in figure 1), we must additionally inspect/acquire the locks for *all* overlapping regions in-order. If the all neighboring regions are evicted or unlocked, the edge-frame can be evicted. If another

Algorithm 2 Evicting a contiguous run of regions

```
1:  function EVICT_CONTIGUOUS_REGIONS(range)
2:      for each frame f in range do
3:          if f is an edge frame then
4:              rs ← LOOKUP_REGIONS_FROM_FRAME(f)
5:              if not LOCK_ALL_REGIONS(rs) then
6:                  if f has no reader then ▷ another evictor
7:                      add f to graveyard
8:                  end if
9:                  ▷ lock held by reader: no need to evict f
10:                 continue
11:             end if
12:         end if
13:         if f has pending I/O then
14:             add f to graveyard
15:             continue
16:         end if
17:         EVICT_FRAME(f)
18:     end for
19: end function
```

Algorithm 3 Retrying graveyard frames

```
1:  function EVICT_GRAVEYARD_FRAMES
2:      sort and deduplicate graveyard
3:      for each frame f in graveyard do
4:          rs ← LOOKUP_REGIONS_FROM_FRAME(f)
5:          if LOCK_ALL_REGIONS(rs) succeeds then
6:              EVICT_FRAME(f)
7:              remove f from graveyard
8:          end if
9:          release region locks
10:     end for
11: end function
```

Algorithm 4 Evicting a frame

```
1:  function EVICT_FRAME(f)
2:      remove f from its mapping PTE
3:      rs ← LOOKUP_REGIONS_FROM_FRAME(f)
4:      for each region r in rs do
5:          INCREMENT_VERSION(r)    ▷ TLB invalidation
6:      end for
7:      INSERT_INTO_FRAME_FREELIST(f)
8:  end function
```

reader is accessing an overlapping region, eviction is skipped for the edge-frame. Otherwise, we delay the reclamation of the edge-frame by placing it in a frame graveyard.

For certain workloads, a simplified option would be to ignore the eviction of edge-frames. This can result in frame exhaustion, but also improve performance, as edge-frames belong to multiple regions, and can be used to mask latency.

Partial Eviction of Regions. Our design guards access to unloaded memory via page faults instead of a blocking `pin` interface, allowing our design to support partially evicting the pages of an in-memory region. This uniquely allows our design to mask I/O latency by allowing the application to begin accessing individual pages in the region before the entire region is loaded.

Partial eviction can also occur due to pending I/O or the concurrent eviction of neighboring regions.

The Pending-I/O Problem. Because our design supports partial eviction, readers may release the lock for regions which still have pending I/O requests. However, this presents a problem, whereby the eviction policy attempts to evict a region which has not been loaded yet.

Frames which still have outstanding I/O requests cannot be reused, as external resources (e.g., NIC, disk) are still accessing them via direct memory access (DMA) using their physical address. This is a consequence of full zero-copy design, where we do not use temporary buffer frames for I/O.

We delay evicting these frames until the DMA access is complete by placing the frame in the graveyard.

Graveyard. We use a graveyard to handle *frames* that could not be evicted with their corresponding region(s). This allows our design to eagerly evict the majority of frames in a region, increasing resilience to straggling I/O requests. Additionally, it allows our design to handle concurrent evictions to neighboring regions, which may share pages.

Our graveyard stores entries of $\langle frame_id, VMA_id \rangle$. At the start of every eviction procedure, eviction is attempted for all frames in the graveyard.

The graveyard eviction procedure is described in algorithm 3. Similarly to in algorithm 2, our design needs to handle frames which belong to multiple regions. Since concurrent evictions may place a frame in the graveyard multiple times, the graveyard is de-duplicated to avoid double-freeing a frame.

3.5 Prefetching

While typical buffer managers require that regions be fully in-memory before access, our fault-protected design allows the application to eagerly read into regions which are partially in-memory. Additionally, our design supports the partial eviction of the underlying frames for in-buffer regions (section 3.4).

Typical memory-mapped I/O designs avoid the latency associated with demand-paging by speculatively performing read-ahead prefetching when pages are faulted. Demand-paging latency is further exacerbated if we intend to memory-map a file over the network.

When a region is pinned for access from the evicted or unlocked states, our design prefetches only the required range of frames for the corresponding region. This avoids spurious I/O requests, which create excess frame pressure and load on I/O resources.

3.6 Shared Frame Pool

Our design uses 4 KiB frames, and supports mapping multiple files (VMAs) which share an underlying frame pool.

Since our design directly manipulates the page tables, no contiguous frame allocator is required. The frame allocator can therefore be implemented by a simple free-list of 4 KiB frames.

3.7 Zero-Copy

Our design is able to pass data from the source device (i.e., disk) into the application with without copying it, using a singular locking scheme (section 3.2).

OS Level. Our design ensures that frames with outstanding I/O are not reused and that I/O requests are always aligned, avoiding the need for a bounce-buffer or page caching.

Application Level. Our design avoids a copy from the cache into the application by integrating reference counted buffer handles into the application. The reference counts are tracked by the shared counter (C) in the lock state, and are used to ensure that locked regions are not evicted.

3.8 Efficient TLB Invalidation

Our design also integrates a novel technique for invalidating TLB entries when frames are unmapped. Instead of eagerly propagating TLB entry invalidations when frames are unmapped, our design uses a cooperative approach where readers validate their local TLB entries before accessing virtual addresses. This avoids the overhead of propagating TLB updates to threads which will not access certain memory ranges.

Every thread maintains a local copy of the version counters in the global region directory by storing the last-seen value. These local values are updated on a per-region basis whenever the global lock state is read.

When any page of a region is unmapped, we signal that all TLB entries for that region's virtual address range are invalid by incrementing the version counter in the global region directory. Algorithm 4 describes how this operation is logically performed.

Readers which wish to read from a region must first validate that the thread-local TLB entries for that range are still valid by comparing the observed version with the one stored in the thread-local version array. If the versions differ, the thread must invalidate all TLB entries for that address range before it can continue reading.

Because our version counter reads piggyback off of the atomic lock's compare-and-swap operations, we do not require any additional atomic operations to implement this signaling scheme. While in algorithm 4 we describe the invalidation as separate operation, our design fuses the version counter increment with the evictor's lock release.

Hardware thread migration must be disabled to avoid a scenario where a thread is migrated to a thread with invalid TLB entries after validation.

4 Implementation

Our design is implemented using a modified version of uCache [13] and OSv [9], and integrated with DuckDB (v1.5.2) [17].

While our system can be generalized to any file with non-overlapping logical regions, our prototype implementation primarily focuses on the read-only analytical workload on Parquet files.

During system startup, our implementation reads all the relevant Parquet file footers and builds the global and local region directories. Parquet files optionally include a page index in the footer, which can be cached and used to construct the region directory.

Each Parquet page (roughly equivalent to a column chunk) is modeled as an individual region. Additionally, also we model the bloom filters used for row group skipping as regions.

Since the sizes of bloom filters is highly variable and often smaller than a single frame, we merge multiple adjacent bloom filters into a logical region using the following heuristic. We merge the next adjacent row group's bloom filters if the current region is < 16 KiB and spans fewer than 64 row groups.

5 Evaluation

All benchmarked implementations are run under QEMU/KVM on a Linux 7.2.2 host running on an AMD EPYC 9654P. The attached storage is a KIOXIA CM7-R formatted with ext4, and is passed through to the guest VM using VFIO. All benchmarks use a Parquet TPC-H dataset generated by DuckDB, and use a cache sized as a percentage of the total dataset.

Initialization time, including the time taken to construct the region directories, is excluded from the evaluated runtime.

5.1 Comparison to cache_httpfs

Because our implementation aims to be integrated with a broader solution which caches remote files stored in object storage and accessed over the network, we evaluate our implementation against the popular `cache_httpfs` DuckDB extension (v0.9.1 on DuckDB v1.5.4) [6].

We configure cache_httpfs to cache on-disk files in memory. `cache_httpfs` caches file ranges by partitioning the file into 512 KiB regions, and handles eviction using a LRU policy.

Figure 3 shows the runtime of each TPC-H query on both systems with cold or warm caches. Table 1 shows the geo-

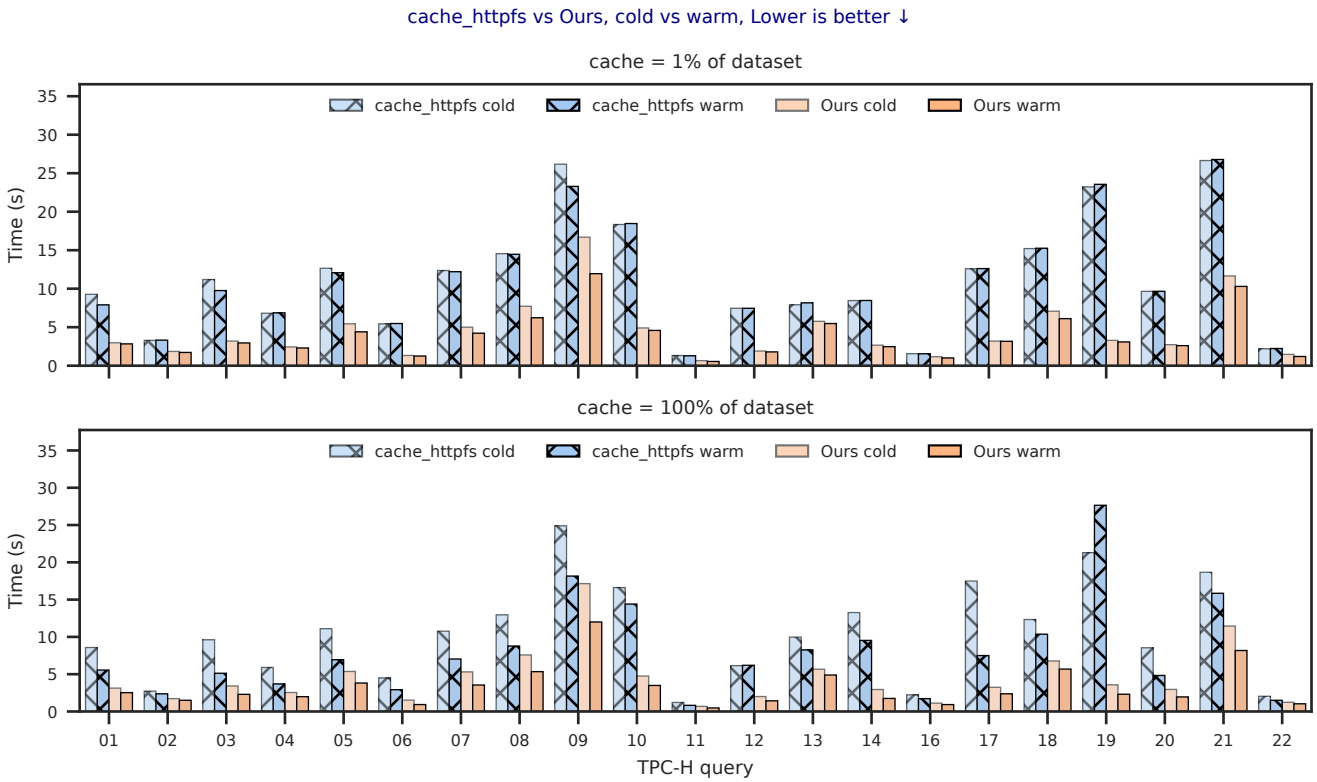


Figure 3: Per-query runtimes for TPC-H SF100 in cold/warm cache scenarios of our implementation and cache_httpfs at 1% and 100% cache sizes using 16 cores.

Cache (% of dataset)	Cold	Warm
1%	2.59×	2.85×
10%	2.49×	2.80×
20%	2.38×	2.90×
40%	2.19×	2.52×
100%	2.43×	2.41×

Table 1: Geo. speedup of our design over cache_httpfs on TPC-H SF100 in cold/warm cache scenarios at various cache sizes using 16 cores.

Step	Cold (s)	Δ	Warm (s)	Δ
base	15.39		15.08	
+region prefetch	4.43	−71%	3.92	−74%
+region evict	4.11	−7%	3.67	−6%
+2 queues	3.99	−3%	3.69	−0%
+no IPI	3.83	−4%	3.57	−3%
+zero-copy	3.73	−3%	3.38	−5%

Table 2: Ablation study of various components of our design in hot and cold runs on TPC-H SF 300 Query 6 at 1% cache size using 16 cores, averaged over 3 runs.

metric mean speedup over `cache_httpfs` over all TPC-H queries.

Our design is faster than `cache_httpfs` on every query, in every tested configuration. Query 19 consistently demonstrates the largest speedup over `cache_httpfs`, ranging between 5.96× and 11.95×. We observe a geometric mean speedup of $> 2\times$ in both warm and cold cache scenarios over `cache_httpfs`.

5.2 Ablation Study

We present an ablation study of the different components of our design on TPC-H SF300 Q6 in warm and cold cache scenarios at a 1% cache size.. The results are shown in table 2.

Baseline. We use a uCache-like memory mapped I/O implementation with a frame-level resident set for eviction.

Region Prefetch. We enable region-aware fetching of file ranges within the VMA before they are read.

Region Evict. We then include region range aware eviction using RegionEntry locking. The eviction policy used at this step scans over the RegionEntry directory using a per-file clock cursor and a second chance policy. We store whether or not a region has been marked for eviction using a marked lock state.

2 Queues. This step replaces the second chance eviction

policy with the two FIFO queue resident set described in 3.3.

No IPI. IPI-based TLB shootdowns are replaced by our IPI-free scheme as described in 3.8.

Zero-copy. We avoid the application-level copy out of our caching VMA. DuckDB must still decompress compressed column chunks into newly allocated buffers, but reads directly from the VMA instead of an intermediary buffer.

A large portion of our speedup over the ablation baseline attributable to region-aware prefetching. The evaluated query (TPC-H Q6) is a pure scan workload, which is an ideal scenario for our baseline's clock based eviction policy.

5.3 Comparison to uCache

Our design extends uCache with file format and workload aware and eviction improvements; we compare our implementation to an unmodified uCache to evaluate their benefit.

We compare the two systems at 5% cache size on TPC-H SF300, because uCache is unable to complete the benchmark at 1% due to a frame pool exhaustion. While the baseline uCache also includes region-aware prefetching, its eviction policy operates on a page frame level.

Across all the queries, our implementation achieves a geometric mean speedup of 1.56× when caches are cold, and 1.21× when caches are warm. In short queries (2, 11, 22), our implementation does not show any noticable speedup over uCache. uCache achieves a faster runtime on a single query, the cold run of query 16, which is a relatively compute heavy query which performs multiple joins, aggregations, and a sort on a small dataset.

5.4 IPI-free TLB invalidation

We evaluate the effectiveness of the IPI-free TLB invalidation mechanism on TPC-H query 6, which is a single table scan and aggregation.

The cache is sized at 2% of the total dataset. We set a limit on the number of frames in an eviction batch, and thus the IPI-invalidation batch size, at 64. We also bound the maximum number of prefetched frames to the same value to avoid buffer exhaustion and excess eviction.

The results of our evaluation are presented in figure 5. Our evaluation shows runtime improvements ranging from $< 1\%$ to 6% over the traditional approach of using inter-processor interrupts (IPI) to invalidate TLB entries when frames are unmapped. The runtime benefit of our approach improves as the number of cores scales, demonstrating that our approach provides a solution to the TLB invalidation bottleneck that other virtual memory-based approached suffer from.

The IPI-based TLB shootdown implementation in OSv requires that the initalising thread holds a lock while waiting for all other cores to acknowledge the TLB shootdown in a spin loop. This serializes frame evictions, as threads need to take turns issuing TLB shootdowns. The collective nature of this TLB shootdown approach means that straddler threads can slow down the entire system.

We additionally observe an interaction where spinning threads are prone to being de-scheduled by the host. While our design requires that the client VM's cores are pinned on the host hardware, we observe in other experiments that a simple `taskset` mapping combined with the traditional TLB shootdown approach results in a spike in the `nonvoluntary_ctxt_switches` scheduler event on the QEMU process, resulting in exceptionally poor performance. Our design does not suffer from similar issues; we also mitigate this effect for the traditional approach in our evaluation by precisely pinning the threads.

We do not evaluate with higher core counts due to limitations with this version of OSv, but we expect the observed pattern to hold, especially on NUMA machines which have increased core-to-core latency.

6 Related Work

Variable sized buffer management. GreedyDual-Size introduces a size-aware cost function for web caching applications [22].

The Umbra database system maps its buffer managed regions multiple times in multiple discrete size classes, but relies on the OS to allocate frames [16].

AdaCache implements a caching scheme with multiple block sizes for file ranges in the context of cloud block storage [21].

Avoiding TLB shootdowns. Both ARM (`TLBI`) [18] and `x86_64` (`INVLPGB`) [1] processor architecture implementations include specialized instructions to broadcast TLB invalidations to all cores. However, these approaches are not portable; Intel does not implement the `INVLPGB` instruction in their processors.

Amit et al., present various optimizations to speed up TLB shootdown routines [2].

Tabby, implemented as part of a broader DB-OS co-design project, intrusively embeds a page identifier in the header of each 4 KiB page [23]. The page identifier is then used to validate that the accessed page did not belong to a stale TLB entry. Our design achieves a similar software-validation of TLB entries using a non-intrusive counter, and supports multi-frame and non-aligned memory regions.

BareHeap tackles the issue of TLB shootdowns by delaying the invalidation of TLB entries until page exhaustion [15].

7 Conclusion

In this report, we demonstrate how semantics-aware DB/OS integration can be used to improve performance over general-purpose state of the art caching systems. We achieve this by designing a locking and eviction scheme which allows

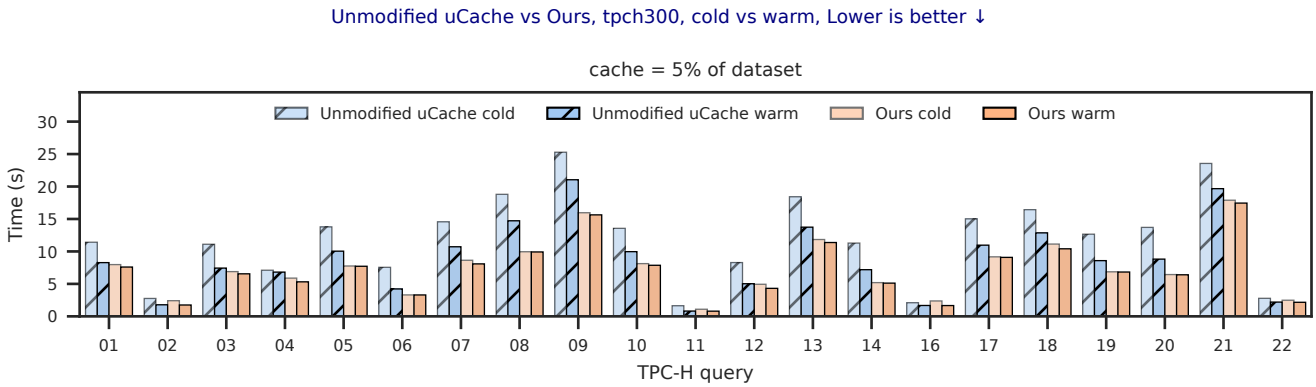


Figure 4: Per-query runtimes for TPC-H SF300 in cold/warm cache scenarios for our implementation and uCache at 5% cache size using 16 cores.

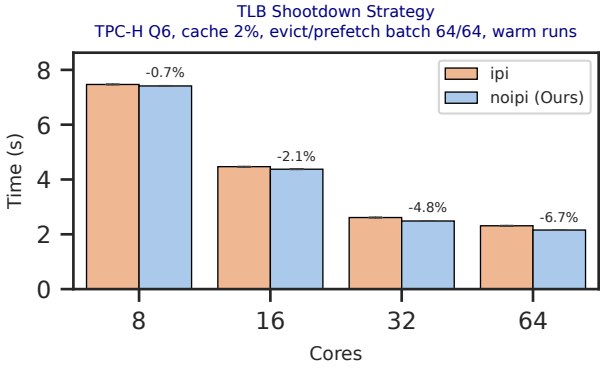


Figure 5: Comparison of IPI-based/free TLB shootdown approaches on TPC-H SF100 Query 6 at various core counts, with maximum prefetch and eviction batch size of 64 frames, averaged over 3 runs.

for variably-sized file regions to be managed using virtual memory. Additionally, we demonstrate a TLB invalidation technique that allows virtual-memory based buffer management to scale to many-core systems.

While our evaluation is limited to analytics workloads, we show promising improvements over existing state of the art caching techniques.

References

[1] Advanced Micro Devices, Inc. *AMD64 Architecture Programmer's Manual, Volume 3: General-Purpose and System Instructions*, revision 3.38 edition, 2026.

[2] Nadav Amit, Amy Tai, and Michael Wei. Don't shoot down TLB shootdowns! In Angelos Bilas, Kostas Magoutis, Evangelos P. Markatos, Dejan Kostic, and Margo I. Seltzer, editors, *EuroSys '20: Fifteenth EuroSys Conference 2020, Heraklion, Greece, April 27-30, 2020*, pages 35:1–35:14. ACM, 2020.

[3] Jan Böttcher, Viktor Leis, Jana Giceva, Thomas Neumann, and Alfons Kemper. Scalable and robust latches for database systems. In Danica Porobic and Thomas Neumann, editors, *16th International Workshop on Data Management on New Hardware, DaMoN 2020, Portland, Oregon, USA, June 15, 2020*, pages 2:1–2:8. ACM, 2020.

[4] Matthias Brantner, Daniela Florescu, David A. Graf, Donald Kossmann, and Tim Kraska. Building a database on S3. In Jason Tsong-Li Wang, editor, *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2008, Vancouver, BC, Canada, June 10-12, 2008*, pages 251–264. ACM, 2008.

[5] Andrew Crotty, Viktor Leis, and Andrew Pavlo. Are you sure you want to use MMAP in your database management system? In *12th Conference on Innovative Data Systems Research, CIDR 2022, Chaminade, CA, USA, January 9-12, 2022*. www.cidrdb.org, 2022.

[6] dentiny. duck-read-cache-fs: A DuckDB extension for remote filesystem access cache. https://github.com/dentiny/duck-read-cache-fs, 2025. Accessed: 2026-10-15.

[7] Wolfgang Effelsberg and Theo Härder. Principles of database buffer management. *ACM Trans. Database Syst.*, 9(4):560–595, 1984.

[8] Theodore Johnson and Dennis E. Shasha. 2q: A low overhead high performance buffer management replacement algorithm. In Jorge B. Bocca, Matthias Jarke, and Carlo Zaniolo, editors, *VLDB'94, Proceedings of 20th International Conference on Very Large Data Bases, September 12-15, 1994, Santiago de Chile, Chile*, pages 439–450. Morgan Kaufmann, 1994.

[9] Avi Kivity, Dor Laor, Glauber Costa, Pekka Enberg, Nadav Har'El, Don Marti, and Vlad Zolotarov. Osv - optimizing the operating system for virtual machines. In Garth Gibson and Nickolai Zeldovich, editors, *Proceedings of the 2014 USENIX Annual Technical Conference, USENIX ATC 2014, Philadelphia, PA, USA, June 19-20, 2014*, pages 61–72. USENIX Association, 2014.

[10] Leslie Lamport. Concurrent reading and writing. *Commun. ACM*, 20(11):806–811, 1977.

[11] Viktor Leis, Adnan Alhomssi, Tobias Ziegler, Yannick Loeck, and Christian Dietrich. Virtual-memory assisted buffer management. *Proc. ACM Manag. Data*, 1(1):7:1–7:25, 2023.

[12] Yinan Li, Jianan Lu, and Badrish Chandramouli. Selection pushdown in column stores using bit manipulation instructions. *Proc. ACM Manag. Data*, 1(2):178:1–178:26, 2023.

[13] Ilya Meignan-Masson, Masanori Misono, Viktor Leis, and Pramod Bhatotia. ucache: A customizable unikernel-based IO cache. In André Brinkmann and Philip Shilane, editors, *24th USENIX Conference on File and Storage Technologies, FAST 2026, Santa Clara, CA, USA, February 24-26, 2026*, pages 701–720. USENIX Association, 2026.

[14] Maged M. Michael and Michael L. Scott. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In James E. Burns and Yoram Moses, editors, *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing, Philadelphia, Pennsylvania, USA, May 23-26, 1996*, pages 267–275. ACM, 1996.

[15] Marcus Müller and Viktor Leis. Bareheap: Virtual memory assisted memory allocation for high performance query processing. In *Proceedings of the 22nd International Workshop on Data Management on New Hardware, DaMoN 2026, Bengaluru, India, 31 May 2026- 5 June 2026*, pages 10:1–10:5. ACM, 2026.

[16] Thomas Neumann and Michael J. Freitag. Umbra: A disk-based system with in-memory performance. In *10th Conference on Innovative Data Systems Research, CIDR 2020, Amsterdam, The Netherlands, January 12-15, 2020, Online Proceedings*. www.cidrdb.org, 2020.

[17] Mark Raasveldt and Hannes Mühleisen. Duckdb: an embeddable analytical database. In Peter Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska, editors, *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, pages 1981–1984. ACM, 2019.

[18] Ben Simner, Alasdair Armstrong, Jean Pichon-Pharabod, Christopher Pulte, Richard Grisenthwaite, and Peter Sewell. Relaxed virtual memory in armv8-a. In Ilya Sergey, editor, *Programming Languages and Systems - 31st European Symposium on Programming, ESOP 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings*, volume 13240 of *Lecture Notes in Computer Science*, pages 143–173. Springer, 2022.

[19] Adrian Vogelsgesang, Michael Haubenschild, Jan Finis, Alfons Kemper, Viktor Leis, Tobias Mühlbauer, Thomas Neumann, and Manuel Then. Get real: How benchmarks fail to represent the real world. In Alexander Böhm and Tilmann Rabl, editors, *Proceedings of the 7th International Workshop on Testing Database Systems, DBTest@SIGMOD 2018, Houston, TX, USA, June 15, 2018*, pages 1:1–1:6. ACM, 2018.

[20] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. FIFO queues are all you need for cache eviction. In Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace, editors, *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 130–149. ACM, 2023.

[21] Qirui Yang, Runyu Jin, Ni Fan, Devasena Inupakutika, Bridget Davis, and Ming Zhao. Adacache: A disaggregated cache system with adaptive block size for cloud block storage. In *16th IEEE International Conference on Cloud Computing, CLOUD 2023, Chicago, IL, USA, July 2-8, 2023*, pages 348–359. IEEE, 2023.

[22] Xingjun Zhang, Depei Qian, Dajun Wu, Yi Liu, and Tao Liu. Site-role based greedydual-size replacement algorithm. In Guozhu Dong, Changjie Tang, and Wei Wang, editors, *Advances in Web-Age Information Management, 4th International Conference, WAIM 2003, Chengdu, China, August 17-19, 2003, Proceedings*, volume 2762 of *Lecture Notes in Computer Science*, pages 335–343. Springer, 2003.

[23] Xinjing Zhou, Viktor Leis, Jinming Hu, Xiangyao Yu, and Michael Stonebraker. Practical DB-OS co-design with privileged kernel bypass. *Proc. ACM Manag. Data*, 3(1):64:1–64:27, 2025.